

ПРОГРАМНАЯ ДОКУМЕНТАЦИЯ НА “ОБЛАЧНОЕ
ХРАНИЛИЩЕ КАРТИНОК” РАЗРАБОТАННЫЙ НА ОСНОВЕ

Python

Django

PyQt5

СОДЕРЖАНИЕ

1.Введение	3
2.Техническое задание	3
3.Схема разработанной программы	4
4.Перечень и нумерация событий	4
5.Перечень и нумерация входных переменных	4
6.Системонезависимая часть	5
7.Перечень и нумерация выходных воздействий	5
8.Как пользоваться вашей программой	5
9.Листинг	5

1. ВВЕДЕНИЕ

Проект “Облачное хранилище картинок” включает в себя две части

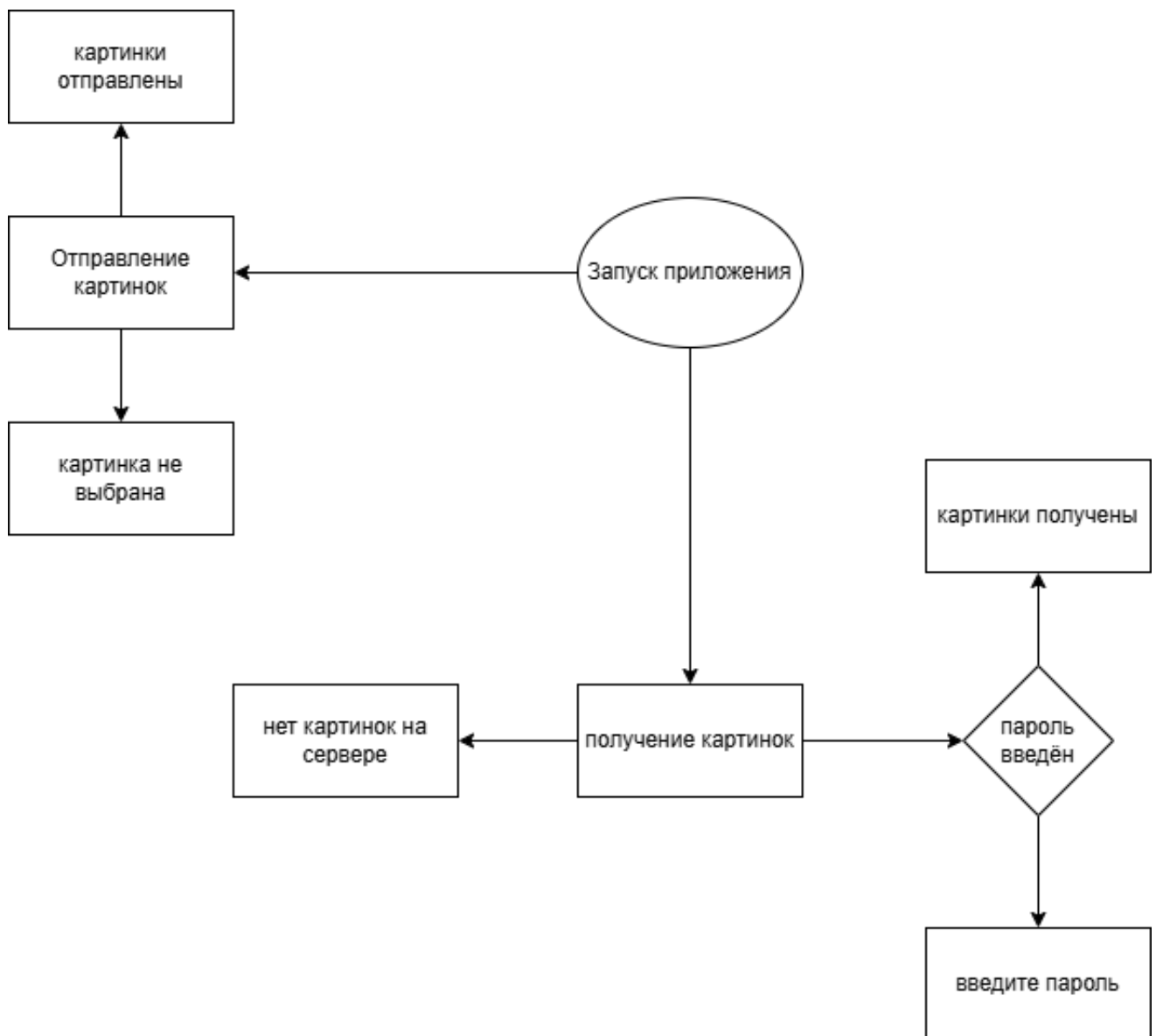
первая является клиентским приложением основанным на библиотеке Pyqt5 для создания интерфейса и позволяет пользователю взаимодействовать с серверной частью проекта перенаправляя по сети файлы изображений на хранение на сервер а также получать с сервера файлы сохранённые там

вторая является серверным приложением принимающим только post и get запросы post для картинок отправляемых клиентом get для отправки картинок клиенту также на сервере настроена примитивная аутентификация с помощью пароля отправляемого клиентом вместе с запросами

2. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Отсутствует

3.СХЕМА РАЗРАБОТАННОЙ ПРОГРАММЫ



4. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ

1. Запуск сервера и клиента
2. Отправление картинок на сервер
3. Получение картинок

5.ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ

1. файл для отправки
2. Пароль для доступа к серверу

6.СИСТЕМОНЕЗАВИСИМАЯ ЧАСТЬ

Клиентское и серверное приложения работают системонезависимо

7.ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ

1. При отправке на сервер клиент уведомляет об успехе
2. При получении картинок с сервера клиент отображает полученные картинки

8.КАК ПОЛЬЗОВАТЬСЯ ВАШЕЙ ПРОГРАММОЙ

- 1.Запустить exe файл
- 2.Выбрать отправку изображения на сервер
 - 2.1.выбрать изображение для отправки
- 3.Ввести пароль для получения изображений
- 4.Получить изображения

9.ЛИСТИНГ

Код	клиентского	приложения
<pre>import os import sys import requests from PIL import Image from PyQt5.QtWidgets import (QApplication, QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTextEdit, QLabel, QGroupBox, QLineEdit, QFileDialog, QListWidget, QListWidgetItem) from PyQt5.QtGui import QImage, QPixmap import json from PIL import Image from PyQt5 import QtCore class MyWindow(QWidget): def __init__(self): super().__init__() self.initUI() def initUI(self): over_main_layout = QVBoxLayout() main_layout = QHBoxLayout() over_main_layout.addLayout(main_layout) self.button1 = QPushButton(f"отправить") self.button2 = QPushButton(f"получить") self.button1.clicked.connect(self.change_text1) self.button2.clicked.connect(self.change_text2) self.images_group = QListWidget() self.images_group.itemClicked.connect(self.item_clicked) self.image = QLabel() self.image_button = QPushButton() self.image_button.setText('выбрать изображение для отправки') self.image_button.clicked.connect(self.pick_file) self.widget3 = QLabel()</pre>		

```

self.security = QGroupBox("введите пароль - (1)")
self.security_layout = QVBoxLayout()
self.security.setLayout(self.security_layout)
self.password_field = QLineEdit()
self.security_layout.addWidget(self.password_field)
self.security_layout.addWidget(self.widget3)
self.security_layout.addWidget(self.image_button)
self.security_layout.addWidget(self.images_group)
self.security_layout.addWidget(self.image)
self.password = '1'
# self.widget1 = self.create_widget("отправить на сервер", 1)
# self.widget2 = self.create_widget("получить с сервера", 2)

self.pixmap = QPixmap('')

main_layout.addWidget(self.button1)
main_layout.addWidget(self.button2)

over_main_layout.addWidget(self.security)
self.setLayout(over_main_layout)
self.setWindowTitle('Два виджета с кнопками и текстом')
self.setGeometry(300, 300, 600, 600)

def create_widget(self, title, widget_num):
    group_box = QGroupBox(title)
    layout = QVBoxLayout()
    if widget_num == 1:
        self.text_edit1 = QTextEdit()
        self.text_edit1.setPlaceholderText(f"Текст для {title}")
        layout.addWidget(self.text_edit1)
    else:
        self.text_edit2 = QTextEdit()
        self.text_edit2.setPlaceholderText(f"Текст для {title}")
        layout.addWidget(self.text_edit2)

    button = QPushButton(f"Изменить текст в {title}")

    if widget_num == 1:
        button.clicked.connect(self.change_text1)
    else:
        button.clicked.connect(self.change_text2)

    layout.addWidget(button)
    group_box.setLayout(layout)

    return group_box

# , 'password' : self.password_field.text()}
def change_text1(self):
    try:
        if self.password_field.text() != self.password:
            self.widget3.setText('введите пароль')

```

```

        return None
    with open (self.image_path[0], 'rb') as f:
        print(f)
        print('-----')
        r = requests.post("http://127.0.0.1:8000/main/images/", files =
{"image": f})
        self.widget3.setText('изображение отправлено')
        print(r.text)
    except:
        self.widget3.setText('выберите изображение для отправки')
        # self.widget3.setText(r.text)

def change_text2(self):
    r = requests.get("http://127.0.0.1:8000/main/images/")
    try:
        if not json.loads(r.content):
            self.widget3.setText('на сервере пусто')
            r1 = requests.get(json.loads(r.content)[0]['image'])
    except:
        self.widget3.setText('ошибка получения(вероятно на сервере пусто)')
    for i in json.loads(r.content):
        x = requests.get(i['image'])
        self.pixmap.loadFromData(x.content)
        scaled_pixmap = self.pixmap.scaled(200, 200, QtCore.Qt.KeepAspectRatio)
        self.image.setPixmap(scaled_pixmap)
        QListWidgetItem(i['image'], self.images_group)
        # self.text_edit2.setText(r.text)

def item_clicked(self, item):
    print(item.text())
    x = requests.get(item.text())
    self.pixmap.loadFromData(x.content)
    scaled_pixmap = self.pixmap.scaled(200, 200, QtCore.Qt.KeepAspectRatio)
    self.image.setPixmap(scaled_pixmap)

def pick_file(self):
    self.image_path = QFileDialog().getOpenFileName(
        self,
        'выберите изображение',
        '',
        'image (*.png *.jpg)'
    )
    self.widget3.setText('изображение выбрано')
    # self.image = Image.open(fp=self.image_path[0])
    # print(self.image)
    # self.image.setNameFilter("image (*.png *.jpg)")

if __name__ == '__main__':
    os.system('python project/run.py')
    app = QApplication(sys.argv)
    window = MyWindow()
    window.show()
    sys.exit(app.exec_())

```

Модели

данных

сервера

```
from django.db import models
```

```
# Create your models here.
```

```
class Image(models.Model):
```

```
    image = models.ImageField(upload_to='images/')
```